

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

Институт информационных технологий и анализа данных
Кафедра информационных технологий и систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

по дисциплине «Программирование»

Лабораторная работа 4. Проверка условий

Студент
гр. БИС-26-3

Н.М. Единархов

Преподаватель

М.В. Водяницкий

1 Цель работы

Освоить условный оператор `if-elif-else` в языке Python, операторы сравнения, логические операторы `and` и `or` и остаток от деления `%`. Научиться проверять ограничения на входные данные и выбирать результат по набору условий без использования циклов, коллекций и исключений.

2 Задание 1. Управление кондиционером

Постановка задачи. Пользователь вводит текущую и желаемую температуру как два вещественных числа на отдельных строках. Необходимо вывести режим работы кондиционера: «Охлаждение», если текущая температура выше желаемой, «Нагрев», если ниже, и «Выключен», если температуры равны.

Описание решения. Значения считываются функцией `input()` и преобразуются в вещественные числа функцией `float()`. Три возможных результата взаимоисключающие, поэтому для выбора используется одна цепочка `if-elif-else`. Сначала проверяется условие `current > desired`, затем `current < desired`. Если ни одно из них не выполнено, температуры равны, и это обрабатывает ветка `else`.

Текст программы:

```
current = float(input())
desired = float(input())
if current > desired:
    print('Охлаждение')
elif current < desired:
    print('Нагрев')
else:
    print('Выключен')
```

Результат работы программы для входных данных 27.5 и 23 показан на рисунке 1.

```
1  a = float(input("Введите текущую температуру:"))
2  b = float(input("Введите желаемую температуру:"))
3  if a > b:
4      print("Охлаждение")
5  elif a < b:
6      print("Нагрев")
7  else:
8      print("Выключен")
9
10
11
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\User\AppData\Local\Programs\Microsoft VS Code> &
y
Введите текущую температуру:52
Введите желаемую температуру:23
Охлаждение
```

Рисунок 1 – Результат работы программы задания 1

3 Задание 2. Расчёт стоимости билета

Постановка задачи. Пользователь вводит базовую стоимость билета как вещественное число и возраст пассажира как целое число на отдельных строках. Стоимость должна быть положительной, возраст должен находиться в диапазоне от 0 до 120 включительно. При нарушении ограничения необходимо вывести «Ошибка». Для корректных данных цена рассчитывается как доля базовой стоимости: 0% для возраста от 0 до 5 лет, 50% от 6 до 17 лет, 100% от 18 до 59 лет и 70% от 60 до 120 лет. Результат выводится с двумя цифрами после десятичной точки.

Описание решения. Стоимость считывается как `float`, возраст как `int`. Первым в цепочке `if-elif-else` проверяется некорректный ввод: стоимость не больше нуля или возраст вне диапазона от 0 до 120. Так как условия соединены оператором `or`, достаточно нарушения любого из ограничений. Следующие ветки `elif` проверяют возраст по возрастанию границ: до 5, до 17 и до 59 лет включительно. Благодаря порядку проверок нижние границы диапазонов писать не нужно, так как подходящие меньшие значения уже обработаны предыдущими ветками. Ветка `else` обрабатывает возраст от 60 до 120 лет. В каждой ветке цена умножается на нужную долю и выводится в формате `.2f`.

Текст программы:

```
price = float(input())
age = int(input())
if price <= 0 or age < 0 or age > 120:
    print('Ошибка')
elif age <= 5:
    print(f'Стоимость: {price * 0:.2f} руб')
elif age <= 17:
    print(f'Стоимость: {price * 0.5:.2f} руб')
elif age <= 59:
    print(f'Стоимость: {price:.2f} руб')
else:
    print(f'Стоимость: {price * 0.7:.2f} руб')
```

Результат работы программы для входных данных 1800 и 16 показан на рисунке 2.

```
1 price = float(input())
2 age = int(input())
3
4 if price <= 0 or age < 0 or age > 120:
5     print("Ошибка")
6 elif age <= 5:
7     print("Стоимость: 0.00 руб")
8 elif age <= 17:
9     print(f"Стоимость: {price * 0.5:.2f} руб")
10 elif age <= 59:
11     print(f"Стоимость: {price:.2f} руб")
12 else:
13     print(f"Стоимость: {price * 0.7:.2f} руб")
```

Problems Output Debug Console Terminal Ports

PS C:\Users\User\AppData\Local\Programs\Microsoft VS Code> & C:\User:
r\Desktop\для лаб/task2.py"
1800
12
Стоимость: 900.00 руб

Рисунок 2 – Результат работы программы задания 2

4 Задание 3. Определение положения точки

Постановка задачи. Пользователь вводит координаты x и y как два вещественных числа на отдельных строках. Необходимо определить положение точки: начало координат, ось X , ось Y либо одна из четырёх четвертей.

Описание решения. Координаты считываются как `float`. Положение точки определяется одной цепочкой `if-elif-else`. Сначала проверяются точки на осях, потому что у них одна из координат равна нулю: условие `x == 0 and y == 0` даёт начало координат, затем `y == 0` даёт ось X (случай `x == 0` уже исключён предыдущей веткой), затем `x == 0` даёт ось Y . После этого обе координаты ненулевые, и четверть определяется по знакам: I четверть при `x > 0 and y > 0`, II при `x < 0 and y > 0`, III при `x < 0 and y < 0`. Оставшийся случай (`x > 0, y < 0`) обрабатывает ветка `else` как IV четверть.

Текст программы:

```
x = float(input())
y = float(input())
if x == 0 and y == 0:
    print('Начало координат')
elif y == 0:
    print('Ось X')
elif x == 0:
    print('Ось Y')
elif x > 0 and y > 0:
    print('I четверть')
elif x < 0 and y > 0:
    print('II четверть')
elif x < 0 and y < 0:
    print('III четверть')
else:
    print('IV четверть')
```

Результат работы программы для входных данных -4 и 7 показан на рисунке 3.

```
1  x = float(input())
2  y = float(input())
3
4  if x == 0 and y == 0:
5      print("Начало координат")
6  elif y == 0:
7      print("Ось X")
8  elif x == 0:
9      print("Ось Y")
10 elif x > 0 and y > 0:
11     print("I четверть")
12 elif x < 0 and y > 0:
13     print("II четверть")
14 elif x < 0 and y < 0:
15     print("III четверть")
16 elif x > 0 and y < 0:
17     print("IV четверть")
```

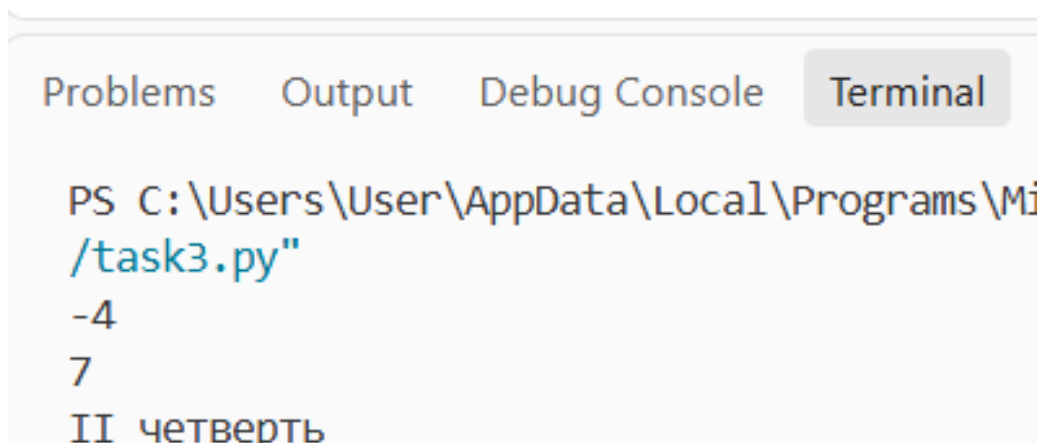


Рисунок 3 – Результат работы программы задания 3

5 Задание 4. Проверка треугольника

Постановка задачи. Пользователь вводит длины трёх сторон как целые числа на отдельных строках. Если хотя бы одна сторона неположительная или найдутся две стороны, сумма которых не больше третьей, необходимо вывести «Треугольник не существует». Для существующего треугольника выводится «Равносторонний», если все стороны равны, «Равнобедренный», если равны ровно две стороны, и «Разносторонний» в остальных случаях.

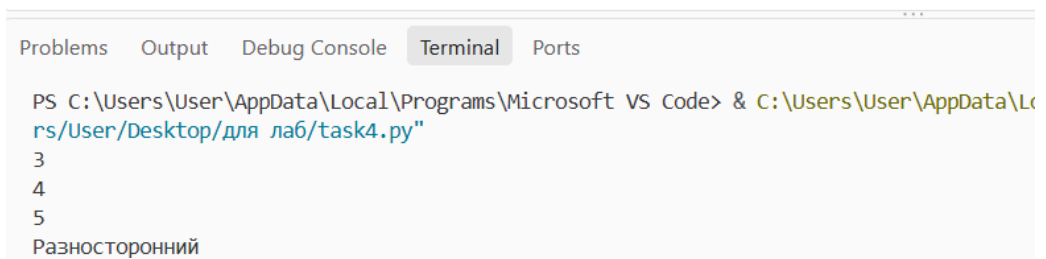
Описание решения. Стороны считываются как `int` в переменные `a`, `b`, `c`. Первое условие цепочки объединяет через `or` все признаки несуществующего треугольника: любая сторона не больше нуля, либо сумма какой-либо пары сторон не больше третьей стороны ($a + b \leq c$, $a + c \leq b$, $b + c \leq a$). Если это условие не выполнено, треугольник существует, и его вид определяют дальше. Условие `a == b and b == c` выполняется только для равностороннего треугольника. Условие `a == b or a == c or b == c` в этой ветке означает равенство ровно двух сторон, так как случай равенства всех трёх уже обработан раньше. Остальные треугольники попадают в ветку `else` и являются разносторонними.

Текст программы:

```
a = int(input())
b = int(input())
c = int(input())
if a <= 0 or b <= 0 or c <= 0 or a + b <= c or a + c <= b or b + c <= a:
    print('Треугольник не существует')
elif a == b and b == c:
    print('Равносторонний')
elif a == b or a == c or b == c:
    print('Равнобедренный')
else:
    print('Разносторонний')
```

Результат работы программы для входных данных 5, 5 и 8 показан на рисунке 4.

```
1  a = int(input())
2  b = int(input())
3  c = int(input())
4
5  if a + b <= c or a + c <= b or b + c <= a or a <= 0 or b <= 0 or c <= 0:
6      print("Треугольник не существует")
7  elif a == b and b == c:
8      print("Равносторонний")
9  elif a == b or a == c or b == c:
10     print("Равнобедренный")
11 else:
12     print("Разносторонний")
```



Problems Output Debug Console Terminal Ports

```
PS C:\Users\User\AppData\Local\Programs\Microsoft VS Code> & C:\Users\User\AppData\Local\Programs\Microsoft VS Code\Code.exe --args C:\Users\User\Desktop\для лаб/task4.py
3
4
5
Разносторонний
```

Рисунок 4 – Результат работы программы задания 4

6 Задание 5. Определение високосного года

Постановка задачи. Пользователь вводит номер года как целое число. Если год не входит в диапазон от 1 до 9999, необходимо вывести «Ошибка». Год является високосным, если он делится на 400 или одновременно делится на 4 и не делится на 100. Для корректного значения выводится «Високосный» или «Невисокосный».

Описание решения. Год считывается как `int`. В цепочке `if-elif-else` сначала проверяется выход за диапазон: `year < 1 or year > 9999`. Затем проверяется условие високосного года: `year % 400 == 0 or (year % 4 == 0 and year % 100 != 0)`. Оператор `%` возвращает остаток от деления, а равенство остатка нулю означает, что число делится нацело. Скобки вокруг условия с `and` показывают порядок вычисления явно. Если условие не выполнено, год невисокосный, и его обрабатывает ветка `else`. Например, 2100 делится на 4 и на 100, но не на 400, поэтому год невисокосный.

Текст программы:

```
year = int(input())
if year < 1 or year > 9999:
    print('Ошибка')
elif year % 400 == 0 or (year % 4 == 0 and year % 100 != 0):
    print('Високосный')
else:
    print('Невисокосный')
```

Результат работы программы для входных данных 2100 показан на рисунке 5.

```
1 year = int(input())
2
3 if year < 1 or year > 9999:
4     print("Ошибка")
5 elif year % 400 == 0 or (year % 4 == 0 and year % 100 != 0):
6     print("Високосный")
7 else:
8     print("Невисокосный")
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\User\AppData\Local\Programs\Microsoft VS Code> & C:\Users\User\Desktop\для лаб/task5.py"
2100
Невисокосный
```

Рисунок 5 – Результат работы программы задания 5

7 Вывод

В ходе лабораторной работы были изучены условный оператор `if-elif-else`, операторы сравнения, логические операторы `and` и `or` и остаток от деления `%`. Все пять программ выполнены без циклов, коллекций и исключений, для выбора взаимоисключающих результатов использована одна цепочка `if-elif-else`. Результаты работы программ совпадают с примерами из задания.